

AI RECRUIT360: FYP DEFENSE MASTER GUIDE

Comprehensive Technical Architecture, Code Mapping & External Examiner Q&A Handbook

Department of Software Engineering | Mirpur University of Science and Technology (MUST)

HOW TO USE THIS HANDBOOK: This guide gives you 100% mastery over your project architecture, algorithms, database operations, and code locations. When an examiner asks a challenging question, stay calm, reference the architectural design decisions outlined below, and cite the exact files and lines of code. This demonstrates that your system is rigorously engineered, highly defensible, and fully understood.

1. High-Level Architectural Flow & Technology Stack

AI Recruit360 is a cloud-native, multi-tenant automated talent acquisition platform structured across three decoupled layers:

- Frontend Layer: Next.js 14 (App Router, React Server Components, TypeScript, Tailwind CSS) deployed on Vercel Edge.
- AI Microservice Layer: Python 3.12 with FastAPI (Asynchronous ASGI runtime, Uvicorn, Pydantic v2 validation) containerized with Docker.
- Database & Storage Layer: Supabase Cloud (Managed PostgreSQL 15, PgBouncer connection pooler on port 6543, S3-compatible Object Storage) hosted in AWS Frankfurt (eu-central-1).
- Multimodal AI Foundations: OpenAI GPT-4o-mini (Screening & Rubrics), OpenAI Whisper-1 (Speech-to-Text), OpenAI TTS Nova (Text-to-Speech), and Simli (Real-Time WebRTC Neural Video Avatar).

Component	Technology / Service	Primary Role	Code Location
Web Portal & Server Actions	Next.js 14, TypeScript	SSR, Auth Cookies, Job/Applicant UI, Candidate Examination Portals	frontend/
AI Microservice Engine	FastAPI, Python 3.12	CV Parsing, Prompt Engineering, Structured LLM Scoring, Interview Evaluation	ai-service/
Relational Database	PostgreSQL 15 on Supabase	3NF Schema, Row Level Security (RLS), ACID transactions, Audit Logs	supabase/migrations/
Object Storage	Supabase Storage	Private encrypted bucket (candidate_documents)	candidate_documents bucket

for uploaded PDF/DOCX
resumes

2. Resume Extraction, Storage, and AI Screening Mechanics

Q1: Exactly how is the resume extracted from uploaded files?

Exact Code Path: The document extraction is handled by an asynchronous multi-tiered parsing engine located in:

ai-service/app/services/cv/extractor.py (functions: `extract_text_from_bytes`, `extract_text_from_pdf`, `extract_text_from_docx`)

- **PDF Parsing:** Uses PyMuPDF (fitz) to open the document stream directly from memory buffer (io.BytesIO). PyMuPDF maintains the logical reading order even in complex two-column resume templates. If PyMuPDF encounters a corrupt stream, it falls back to pypdf. If fewer than 50 characters are detected (indicating a scanned image), it triggers Tesseract OCR fallback.
- **DOCX Parsing:** Uses python-docx via `iter_inner_content()`. Crucially, because modern resume templates place work history and technical skills inside invisible table cells, our parser iterates through BOTH standard paragraphs and table rows/cells, joining text with ' | ' delimiters to ensure zero data loss.
- **Text Sanitization:** Strips non-printable binary artifacts, normalizes Unicode whitespace, and truncates to 24,000 tokens to fit safely within the LLM context window.

Q2: Where is the resume saved? In the database or storage?

Software Architecture Answer: We follow cloud software architecture best practices by separating BLOB (Binary Large Object) storage from relational metadata:

1. The binary file (PDF/DOCX) is uploaded to Supabase Storage in the private bucket 'candidate_documents' under a secure path `[application_id]/[file_name]`. It is never stored directly inside a PostgreSQL table to prevent database bloating.
2. The database stores the pointer and extracted text in the `candidate_documents` table: `id` (UUID), `application_id` (FK), `file_name` (TEXT), `storage_path` (TEXT), `file_type` (MIME type), and `extracted_text` (sanitized text stream).

Code Reference: `frontend/app/actions/applications.ts` (`submitApplication`) and `ai-service/app/repositories/application_repo.py` (`get_candidate_cv_data`).

Q3 (THE BIG QUESTION): Are you using embeddings for CV-Job comparison? Which Vector DB?

CRUCIAL EXAMINER TRAP & WINNING DEFENSE: Many examiners expect students to say 'we used LangChain, embeddings, and Pinecone'. Here is why you should proudly state you **DELIBERATELY** avoided pure vector embeddings for candidate screening, and why your approach is significantly more sophisticated, accurate, and defensible:

Exact Defense Answer to External Examiner: Say this with confidence:

"Sir/Madam, we deliberately chose NOT to rely on vector embeddings and cosine similarity for resume screening, and here is the exact software engineering and mathematical justification:"

1. **The Fatal Flaw of Vector Embeddings in Recruitment:** Vector embeddings (like text-embedding-3 or SentenceTransformers) compress an entire document into a single high-dimensional point in mathematical space. Comparing a resume and job description using cosine similarity only measures vocabulary overlap. For example, if a job requires '5+ years Python, AWS, and PostgreSQL; NO PHP', a candidate with '5 years PHP, 1 year Python' will score a 0.85+ cosine similarity simply because both documents contain web engineering terms. Vector distance cannot enforce boolean prerequisites, cannot evaluate years of seniority, and cannot verify authentic competence.
2. **Complete Lack of Explainability:** Cosine distance returns a single opaque float (e.g., 0.78). It cannot explain WHY the candidate scored 0.78, cannot cite what skills are missing, and cannot provide legal auditability required by modern employment regulations (such as the EU AI Act).
3. **Our Superior Solution — Direct Deep Reasoning with Strict Pydantic Schema Enforcement:** Instead of an opaque vector distance, our pipeline (ai-service/app/services/screening/orchestrator.py) passes both the full job requisition and the candidate's sanitized CV text into OpenAI GPT-4o-mini using strict JSON Schema Enforcement (Pydantic v2 ScreeningDecisionResult). The model performs true multi-dimensional causal evaluation across four criteria:
 - **Technical Skill Alignment (40% weight):** Validates programming languages, libraries, and frameworks.
 - **Experience Level & Seniority (30% weight):** Verifies years of professional tenure and project scope.
 - **Educational & Foundational Background (15% weight):** Validates degrees and certifications.
 - **Soft Skills & Articulation (15% weight):** Analyzes communication clarity and leadership.
4. **Verifiable Text Evidence:** The model is forced to return exact quoted excerpts from the candidate's resume proving where each skill was exhibited. This gives recruiters complete auditability with zero black-box obscurity.
5. **Vector DB Role:** If asked about Vector DBs, mention: 'Supabase PostgreSQL has native pgvector support installed. While pgvector is suitable for coarse semantic search across 100,000+ candidate archives in large databases (Phase 2), for precise evaluation of an applicant against a published requisition, structured LLM reasoning with schema enforcement is demonstrably superior.'

3. Database Architecture, Multi-Tenancy & Data Access

Q4: Where is the database hosted, and how is it accessed from Frontend and Backend?

- **Physical Location:** Supabase Cloud running PostgreSQL 15, deployed in the AWS Frankfurt (eu-central-1) region.
- **Connection Management:** The platform uses PgBouncer connection pooling on port 6543 to manage concurrent database transactions without exhausting PostgreSQL thread pools.
- **Frontend Access (Next.js 14):** Next.js interacts with Supabase using @supabase/ssr inside Server Components and Server Actions (frontend/lib/supabase/server.ts). It passes the user's session JWT token stored in secure, HttpOnly, SameSite cookies. Row-Level Security (RLS) policies are automatically enforced.
- **Backend Access (FastAPI):** The AI microservice accesses Supabase via the official supabase-py SDK (ai-service/app/db/supabase.py). Because supabase-py is synchronous internally, we implemented an asynchronous wrapper run_sync() using asyncio.to_thread() to ensure database I/O calls never block the FastAPI event loop.

Q5: How is multi-tenancy and data security enforced?

- Kernel-Level Row Level Security (RLS): Multi-tenancy is NOT handled in application code with fragile 'WHERE org_id = ...' clauses. Instead, it is enforced directly within the PostgreSQL kernel via Row-Level Security policies. Every table has an organization_id column, and RLS policies verify that auth.uid() exists in the organization_members table with active permissions.
- Role-Based Access Control (RBAC): The organization_members table enforces 5 distinct roles: Owner, Admin, Recruiter, Interviewer, and Viewer. If an Interviewer or Viewer attempts to mutate a job or delete records, PostgreSQL immediately raises an HTTP 403 Forbidden.
- Audit Trail: All security-critical actions (candidate status overrides, deletions, role updates) are recorded in the security_audit_logs table with actor UUID, IP address, and timestamp.

4. Anti-Cheat Assessment & Real-Time AI Avatar Interview

Q6: How does the Server-Authoritative Anti-Cheat Assessment work?

Anti-Cheat Architecture: Commercial testing tools often rely on client-side JavaScript timers (setInterval), which candidates can pause by freezing the browser DOM or modifying localStorage. AI Recruit360 eliminates this vulnerability completely:

1. Dynamic Question Generation: When an assessment starts, OpenAI GPT-4o-mini generates 10 unique, non-trivial multiple choice questions customized to the job requisition (ai-service/app/services/assessment/generator.py).
2. Server-Authoritative Clock: When Question N is served to the candidate, the server records started_at = NOW() in the PostgreSQL database.
3. Strict Server-Side Validation: When the candidate submits an answer, the server computes (submission_time - started_at). If the duration exceeds 50 seconds (45s window + 5s network grace), the answer is automatically invalidated with 0 marks.
4. State Immutability: Once submitted, the answer is permanently recorded in the assessment_answers table with correct_index comparison. The candidate cannot go back, retry, or tamper with previous answers.

Q7: How does the AI Avatar Interview work? What models and protocols are used?

The interview room (frontend/app/interview-room/[application-id]/page.tsx) synchronizes a real-time, low-latency multimodal pipeline across four technologies:

- Video Avatar Gateway: Simli WebRTC streaming API. Simli receives synthetic audio streams from our backend and renders a lip-synchronized, 720p 30fps photo-realistic neural video avatar directly in the candidate's browser viewport.
- Neural Voice Synthesis (TTS): OpenAI TTS-1 (voice: nova) generates natural human prosody with conversational breathing and pacing (ai-service/app/services/interview/tts.py).
- Speech Recognition (STT): The candidate's audio is captured via the browser MediaStream API at 16kHz mono PCM and streamed to OpenAI Whisper (whisper-1) over secure WebSockets (ai-service/app/services/interview/stt.py).
- Candidate Verification & Accessibility: A live transcript appears in the candidate's review panel. The candidate can verify or edit technical terms before submitting. If the candidate has no microphone, the system provides a seamless 'Text Mode' fallback (WCAG 2.1 AA compliance).

- Rubric Scoring: Submitted answers are evaluated by OpenAI GPT-4o-mini against a 4-dimensional rubric: Technical Correctness (40%), Completeness (30%), Depth of Explanation (20%), and Articulation (10%). Results are stored in interview_responses.

5. Multi-Stage Composite Evaluation & Human-in-the-Loop

Q8: How is the final hiring decision made? Does AI make the final hire/reject decision?

Ethical Design Mandate: NO. AI Recruit360 is strictly designed as an Evidence-Based Decision Support System, adhering to international ethical AI standards (such as the EU AI Act).

- Composite Formula: The system calculates a weighted composite score across all three evaluation tiers:

Composite Score = (0.30 * Resume Match) + (0.30 * Assessment Score) + (0.40 * Interview Score)

- Stored in Database: Persisted in final_evaluations alongside sub-scores and qualitative rationales.
- Human-in-the-Loop Oversight: The AI produces an advisory recommendation ('Strong Hire', 'Hire', 'Review', 'Reject'). The platform is programmatically prohibited from executing automated candidate rejections. The human recruiter reviews the candidate dossier (resume quotes, MCQ timestamps, interview transcripts) and retains 100% final hiring authority.

6. Comprehensive File and Code Repository Map

Use this exact directory index to quickly locate any functionality in your codebase during the defense:

Functional Area	Primary Code Files	Key Methods / Classes
Resume Text Extraction	ai-service/app/services/cv/extractor.py	extract_text_from_bytes(), extract_text_from_pdf(), extract_text_from_docx()
AI Resume Screening	ai-service/app/services/screening/orchestrator.py	run_screening_pipeline(), ScreeningDecisionResult Pydantic schema
Assessment Generation	ai-service/app/services/assessment/generator.py	generate_assessment_questions(), AssessmentQuestion schema
Assessment Scoring	ai-service/app/services/assessment/scorer.py	score_assessment(), server- authoritative timestamp verification
Interview Speech (TTS)	ai-service/app/services/interview/tts.py	generate_speech_audio() with OpenAI TTS-1 (nova)
Interview Speech (STT)	ai-service/app/services/interview/stt.py	transcribe_audio_stream() with OpenAI Whisper (whisper-1)
Interview Rubric Evaluation	ai- service/app/services/interview/response_evaluator.py	evaluate_candidate_response() with 4-dimensional rubric
Composite Scoring	ai-service/app/services/evaluation/evaluator.py	calculate_composite_score() [30% CV + 30% Test + 40% Voice]

Database Client & Pooler	ai-service/app/db/supabase.py	get_supabase_client(), run_sync() (async thread pool wrapper)
Application Actions & UI	frontend/app/actions/applications.ts frontend/lib/services/application-service.ts	submitApplication(), getApplicationDetails(), triageCandidate()

7. Top 10 Rapid-Fire Examiner Defense Questions & Answers

Examiner: Why did you use Next.js 14 and FastAPI instead of a single Django or Node.js monolithic app?

Answer: Separation of concerns. Next.js 14 Excel in Server-Side Rendering (SSR), edge routing, and responsive frontend UI, while Python FastAPI is the industry standard for asynchronous machine learning orchestration, PyMuPDF document parsing, and AI model streaming. Coupling them via lightweight RESTful endpoints ensures decoupled scaling and independent deployments.

Examiner: What happens if the network drops while a candidate is uploading their 9 MB resume?

Answer: The upload process is protected by database idempotency and multi-part upload recovery. If an interruption occurs, the candidate's draft application record remains in 'applied' status without corrupting the database. When the candidate retries, the backend detects the existing (job_id, email) record and resumes the session without duplicate rows.

Examiner: How do you prevent candidates from cheating during the technical assessment?

Answer: We use a multi-layered approach: (1) Dynamic Question Generation ensures each requisition has customized, non-static questions. (2) Server-Authoritative Timing: Timers are enforced on the PostgreSQL database server (started_at timestamp), not in client JavaScript. If submitted after 50s, it receives 0 marks. (3) Irreversible submission: Once answered, results are immutably written to assessment_answers.

Examiner: Can an AI model hallucinate candidate skills during screening?

Answer: Yes, LLMs can hallucinate if unconstrained. To eliminate this risk, our screening engine uses Pydantic v2 schemas and prompt-level constraint engineering that forces the model to cite verbatim source quotes from the resume for every claimed skill match. If no quote exists, the skill match is rejected. Furthermore, structured retries are performed at low temperature (0.1).

Examiner: How do you handle candidates with regional accents or speech impediments during the avatar interview?

Answer: We implement three accessibility safeguards: (1) OpenAI Whisper foundation model handles diverse global accents with low Word Error Rates (<8%). (2) Candidates see a real-time review panel where they can manually edit or correct any misrecognized technical terminology before final submission. (3) WCAG 2.1 AA compliant Text Input Mode allows typing answers if audio input is unavailable.

Examiner: Why did you choose Supabase instead of self-hosted PostgreSQL?

Answer: Supabase provides an enterprise-grade managed PostgreSQL 15 environment with integrated Row-Level Security (RLS), real-time WebSockets, automated Write-Ahead Logging (WAL) daily backups, and built-in PgBouncer connection pooling, reducing DevOps overhead while guaranteeing enterprise-grade ACID reliability.

Examiner: What is the computational cost of running this platform?

Answer: As demonstrated in our Chapter 7 economic analysis, fixed baseline infrastructure (Vercel Pro, Render Standard, Supabase Pro) costs ~\$71/month. The variable cost per candidate across the full 3-stage funnel (CV screening + 10 MCQs + 5 avatar interview questions) is only \$0.151 USD, representing a 97% cost reduction compared to traditional recruitment agency fees.

Examiner: What are the United Nations SDGs mapped to this project?

Answer: AI Recruit360 directly addresses five UN Sustainable Development Goals: SDG 8 (Decent Work & Economic Growth via objective competency hiring), SDG 9 (Industry, Innovation & Infrastructure via modern multimodal architecture), SDG 10 (Reduced Inequalities by stripping demographic PII during initial screening), SDG 16 (Peace, Justice & Strong Institutions via immutable audit logs), and SDG 4 (Quality Education by validating technical competencies).

Examiner: How does your composite evaluation weight different stages?

Answer: We utilize a research-backed weighted formula: 30% Resume Match (document qualifications), 30% Server-Timed Technical Assessment (objective problem solving), and 40% Conversational Interview (depth of explanation, technical articulation, and problem breakdown).

Examiner: What is the main future enhancement planned for AI Recruit360?

Answer: Implementing self-hosted open-weight LLMs (Llama 3 8B / Mistral 7B) using vLLM for completely air-gapped enterprise deployments, adding containerized WebAssembly coding sandboxes (Pyodide) for live in-browser coding tests, and supporting multilingual interviewing in Urdu and Arabic.